Cloud-Based Scalable Data Sanitisation & Validation Service and External API Integration

Abstract— This Data sanitization and validation service project presents a cloud-based data sanitization and validation service designed to enhance the reliability of structured data. This system allows users to upload a dataset file and analyses them to identify the common data mistakes like missing values, invalid email formats, twin records, error time related information. The system generates a health score that considers the overall quality of the dataset, but the health score depends on the checks.

The solution is designed to use scalable cloud architecture with Amazon Web Service services like API Gateway and Lambda these are enabling the system to countless requests efficiency without relaying on fixed server architecture. That design confirms flexibility, scalability, enhanced performance under modified workloads.

To improve validation accuracy, the system combined external APIs. The public API is an email validation API that is used to verify the exactness of email addresses, enhancing the reliability of user related data. And another API is Time conversion API. This is developed by a classmate this is used to validate and standardize timestamps including time zone data, assuring consistency across datasets.

These results demonstrate which the system can efficiently detect and report data quality issues meanwhile providing proper insights for data improvement. Overall, the project shows how cloud computing and API integration can be integrated to build a scalable and practical data quality solution for real world applications.

Introduction

Current days digital world, a huge amount of data is collected and used in different applications. Still, not all data is clean. Most of the datasets contain issues like missing values, wrong email formats, copy of the records, and inaccurate time information. Low-quality data can direct wrong decisions, system errors, and reduced performance of applications. Accordingly, it is important to check and enhance data quality before using it.

The main purpose of this project is to solve the problem by developing a simple scalable data sanitization service. The purpose is to help users quickly analyze the problems in their data and enhance their reliability. Rather than manually checking quality of data via a health score.

This service is developed using Cloud technologies to assure scalability and flexibility. Using services like API Gateway and Lambda, the system can manage multiple requests efficiently without depending on the fixed server. This creates the solution meet for real world applications at data size and usage can increase over time.

Moreover, this project combined external APIs to enhanced validation accuracy. An email validation API is used to check whether email address is valid, time conversion API is used to verify and standardize time related data. Those integrations improved the overall quality of the analysis.

The primary goal of this project is to develop a cloud-based system this can analyze data, detect common errors, and give useful perspectives to enhance data quality in a simple and efficient way.

Project Specification and Requirements

This Project Specification is on designing and implementing a cloud-based data sanitization service that can be analyzed and enhance the quality of structured data. The system needs to accept user input data, prepare and restore a clear summary of data quality problems.

Functional Requirements:

The system provides the following functions:

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

- Accept the user input data between a frontend interface (CSV, JSON or TXT input)
- Validate the data and identify the common issues such as:

  Missing values

  Invalid email formats

  Duplicates of records

  Incorrect or missing time and time zone values
- Create a data quality summary including:

  Total records

  Number of errors

  Health score
- Combined email Validation API to improve performance of email checks
- Combined a time conversion API for validating time related data
- Display the results clearly on the UI

Non-Functional Requirements:

- **Scalability:** The system must handle multiple user requests efficiently using cloud services
- **Performance:** Data processing must be fast and responsive for user interaction.
- **Usability:** The UI must be simple and easy to use for users.
- **Reliability:** The system must consistently return correct results for valid inputs.

Cloud Requirements:

- **API Gateway:** This is used for handling the incoming requests from the frontend.
- **Lambda**: AWS Lambda is used for processing data and performing validation Logic.
- **AWS DynamoDB**: DynamoDB is used to store processed data and results, enabling scalable and fast data.
- **CloudWatch**: This is used for monitoring, logging, and debugging the system activity.

The architecture allows system to scale directly and managing increasing workloads without any manual intervention.

Code Quality & Security Requirements:

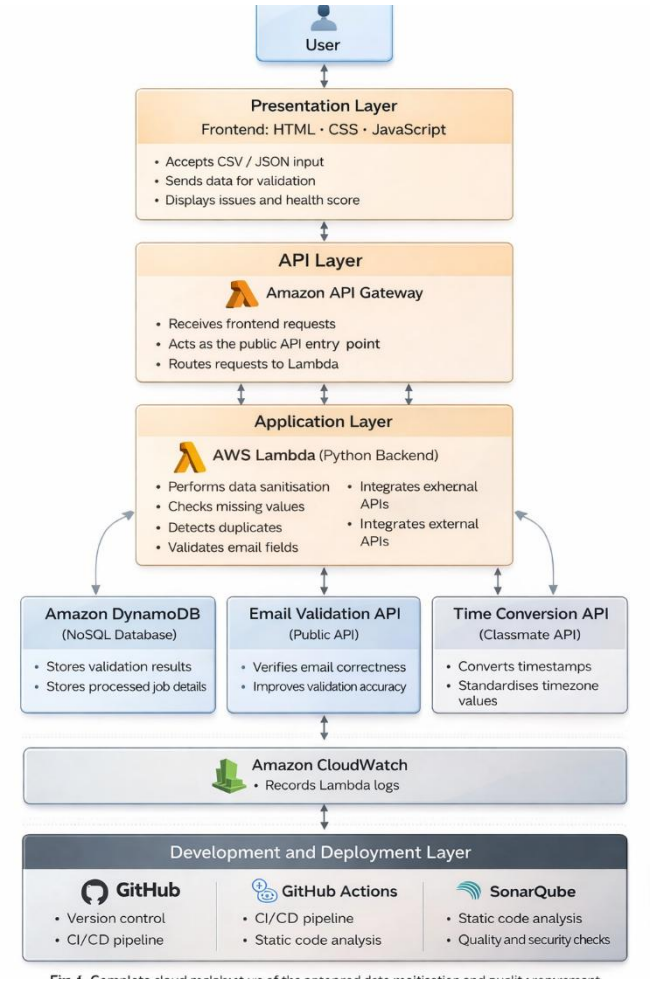
This is used to improve the code quality and maintenance; the project uses SonarQube:

- This detects the code issues like code bugs and code smells
- Also identifies potential security vulnerabilities
- This helps to maintain clean and readable code
- Supports better software development practices.

This ensures the application not only functional but also secure and well structured.

Architecture & Design Aspects of the Application:

This service is designed as a cloud-based scalable service to enhance quality of structured data. This selected architecture uses directed AWS services so the system can be processing requests with efficiency, scale when it is needed, and reduce the need for manual server management.



In this architecture, the frontend is allows users to upload CSV, JSON or TXT data and view validation results. This API Gateway acts as the entry point and forwards requests to Lambda, whenever the main data processing and validation logic is executed. These results are stored in DynamoDB, meanwhile logs are recorded in CloudWatch for monitoring.

The system also integrates two other APIs:

- An email validation API is used for verifying email correctness.
- And a time conversion API for standardising timestamp and time zone data.

Along as well, a development and deployment layer also included, consisting of GitHub, GitHub actions and SonarQube that is supports CI/CD for code quality analysis.

B. Justification of the Architecture:

The Architecture is suitable for this service for several reasons.

- API Gateway is providing a structured and secure way to reveal the backend services. That allows the frontend to communicate with the backend by using HTTP requests and assures proper routing and handling the requests.
- Lambda supports a serverless design where processing happens only when the request is received from UI. That is the ideal for a data validation service because it avoids running a constant server and improves cost efficiency.
- AWS DynamoDB is used for the Storage because it supports fast, scalable and flexible for data storage. After input data can vary in structure, a NOSQL database is more suitable compared to relational database.
- The combing of External APIs improved the system's functionality. An email validation API enhances the accuracy of email checks, whereas the time conversion API assures the consistent timestamp and time zone validation. These combinations make the system more practical for real world use.
- CloudWatch is used for monitoring and logging. This helps to track the system behaviors, support debugging and detects errors, that is improves reliability.
- The integration of GitHub, GitHub actions and SonarQube is supports the CI and code quality. This is assures the application is not only functional but also maintainable and secure.

C. Design Aspects of the Application:

This application follows a layered and modular design approach. Every layer performs a specific function:

- Presentation layer handles the user interaction.
- API layer manages request routing.

- Application layer processes and validate data.
- Data layer stores results.
- Monitoring layer tracks system behavior.
- Development layer manages code and deployment

This responsibilities of separation improves maintainability and allows independent updates to different parts in the system.

Data validation rules are used to develop backend logic:

- For checking the missing values
- Identify the duplicate records
- Validating the email formats
- Validates the timestamp and time zone data

D. Design Patterns used:

The serverless event driven pattern is used, where Lambda is triggered only when a user sends a request. This reduces the resource usage and improves the scalability.

This system is also uses a API based integration pattern where external API services as email validation API and time conversion API are used to enhance functionality without building everything from starting.

Also, a modern design pattern followed where components such as frontend, backend, storage, and monitoring are separated. This enhances flexibility and makes the system easier to maintain.

E. Critical Analysis:

The architecture is having several strengths. That is Scalable, AWS managed services can Manges the multiple requests with efficiency. This is also cost-effective, since serverless computing uses resources only when that is needed.

The system is flexible and allowing easy updates and improvements. External API combination is also enhances the functionality and makes the system more practical.

Still there are some limitations. The use of multiple cloud services raises complexity, making debug more difficult. The system also depends on external APIs, that may affect performance if they are slow.

F. CI/CD in the design & Role of SonarQube:

CI/CD and SonarQube are not a part of the runtime but they play an important role in development.

GitHub is used for version control, GitHub actions are used for supporting the CI/CD automation, SonarQube is used for static code analysis. This tool helps to identify the bugs, enhance the code quality, and ensure better security.

G. Summary:

This architecture provides a scalable, flexible and efficient solution for data sanitization. The serverless computing and modular design enhances maintainability when the external APIs enhances the validation accuracy.

Also, the system introduces the some complexity and risks of dependency, it is well suited for modern cloud-based applications.

Implementation:

Implementation of this system is carried out using cloud services and web technologies. The main goal was to develop a working data sanitization service that can accept user input, process it and return proper results.

A. ***Frontend:***

The frontend of this application is developed by using HTML, CSS, and JS. These are provides a interface where users can upload data in CSV, TXT or JSON format.

- It accepts the user input
- Converts data into a suitable format

- And also sends the data to the backend using the API request
- And displays the results as errors and health score.

B. *AWS API Integration:*

Communication between the front end and back end is manages using API Gateway.

When the user submits data:

- The frontend sends a POST request to the API Gateway.
- AWS API Gateway receives the request
- That is forwards the request to Lambda.

C. *Implementation Of Lambda:*

The main processing of the application is designed by using Lambda with python.

- This is receives input data from API Gateway
- It reads and processes each record
- Checks for missing values
- Including validates email format
- Detects duplicate records
- Validates time and time zone data
- And calls external APIs when needed

After processing, this function calculates:

- Total records
- Total number of errors
- Total health score

All these results are returned to the frontend.

D. *External API Integration:*

The two external APIs is used to enhance validation accuracy.

- The email validation API is checks whether email addresses are correct and usable.
- Time conversion API is used to convert timestamps and time zone values.

This APIs help enhance the quality of validation without adding complex logic to the system.

E. *Inpmlementioan of DynamoDB:*

The processes data and results are stored in DynamoDB.

- Store the validation results
- Store the processed job details

F. *CloudWatch (Monitoring):*

CloudWatch is used monitoring and logging

- Lambda execution logs
- Errors and system activity

G. *CI/CD and code Quality:*

This project uses GitHub, GitHub Actions, SonarQube for deployment and development

- GitHub is used for store and manages the source code
- GitHub Actions are used to automate development and deployment steps
- SonarQube is used to analyze quality of code and identify the issues

H. *Implementation summary:*

This system was successfully enhanced using a combination of frontend and cloud services. Frontend cllects user input, the backend Processes and validates the data, the results are stored and displayed to the user.

The use of services from AWS, external APIs, CI/CD tools ensures that the system is scalable,reliable and easy to maintain.

CONTINOUS INTEGRATION, DELIVERY AND DEPLOYMENT

This system uses a CI/CD process to handle code updates, testing, and deployment. That helps enhance the code quality, reduce errors, and make the development process faster and more reliable.

A. *Continuous Integration:*

CI is used to assure that code changes are tested and checked automatically.

- The GitHub is used for storing the source code
- Every time code is updated or pushed, changes are tracked

- GitHub Actions is used to directly run tasks as build and check

This process helps to identify the issues early and ensures that the code remains stable.

B. Code Analysis and Quality:

Improve code quality, the projects use SonarQube.
SonarQube performs:

- Static code analysis
- Identify of the bugs and code smells
- Identification of security issues

C. **Continuous Deployment and Delivery**:

CI/CD is used to automate the process of deploying the application

- GitHub Actions are used to automate deployment steps
- When the code is updated, the application can be build and deployed automatically
- This is decreases the manual effort and saves time

Backend services like Lambda can be updated using the pipeline, assuring that the latest version of the application is available always.

D. Benefits Of CI/CD:

The CI/CD provides several benefits:

- Improves the code quality
- Decreases the manual errors
- Speed up development and deployment
- Supports the continuous improvement

E. Summary:

This CI/CD process helps maintain a reliable and efficient development workflow. Using of GitHub, GitHub Actions, and SonarQube, this project ensures that code is already tested , analysed, and deployed in a proper structured way. This is improves the total quality and stability of the application.

Conclusion:

The Project is successfully designed a cloud based data sanitisation service this checks the quality of the data by identifying the problems like missing values, duplicate records, invalid emails and incorrect time related information. This provides a health score to help users to understand the data quality.

One key finding is that cloud services such as AWS API Gateway, Lambda, and DynamoDB is make it simpler to develop scalable and flexible applications without maintaining the servers. The use of external APIs like email validation API and time conversion API , these are improves the accuracy and unusefulness of the system.

In development time, challenges are faced in configuring the cloud services and debugging across different components. Tools like CloudWatch is helped in identifying and resolving issues. The SonarQube use is enhance the code quality by detecting bugs and security issues.

This project overall is enhancing to understand of cloud computing, serverless architecture, and API integration. This is demonstrates the a scalable and practical data quality service can be built using cloud technologies, that also highlighting areas for future improvements.

References:

[1] Amazon Web Services, "Amazon API Gateway," [Online]. Available: https://aws.amazon.com/api-gateway/

[2] Amazon Web Services, "AWS Lambda," [Online]. Available: https://aws.amazon.com/lambda/

[3] Amazon Web Services, "Amazon DynamoDB," [Online]. Available: https://aws.amazon.com/dynamodb/

[4] Amazon Web Services, "Amazon CloudWatch," [Online]. Available: https://aws.amazon.com/cloudwatch/

[5] SonarSource, "SonarQube Documentation," [Online]. Available: https://www.sonarsource.com/products/sonarqube/

[6] GitHub, "GitHub Actions Documentation," [Online]. Available: https://docs.github.com/en/actions

[7] MDN Web Docs, "Fetch API," [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

[8] OWASP Foundation, "OWASP Secure Coding Practices," [Online]. Available: https://owasp.org/www-project-secure-coding-practices/